

People's Democratic Republic of Algeria
Ministry of Higher Education and Scientific Research
National Higher School of Cybersecurity — NHSCS
Algorithms and Data Structures — ALSDD

Project Technical Report

History of Algeria Database Manager *A Multi-Module C Application with GTK-3 GUI*

Academic Year : 2025 / 2026
Module : Algorithms and Data Structures (ALSDD)
Institution : NHSCS — Sidi Abdallah, Algiers

Project Team Members :

1. FOUNAS Ouail — Group A4
2. KEBBICHE Anes — Group B1

Submitted as part of the Algorithms and Data Structures module — Academic Year 2025/2026

1. Project Overview

This report documents the design and implementation of the History of Algeria Database Manager, a console and GUI application developed in C for the Algorithms and Data Structures (ALSDD) module at the National Higher School of Cybersecurity (NHSCS). The project implements four fundamental data structure modules — Linked Lists & Queues, Stacks, Binary Search Trees, and Recursion — each applied to a real dataset of Algerian historical personalities and events.

The application supports full CRUD operations (Create, Read, Update, Delete) on the dataset through an interactive terminal interface, complemented by a graphical user interface built with GTK-3. Data is persisted in a plain-text file (data.txt) whose format is parsed at startup into the appropriate in-memory data structure for each module.

Component	Detail
Programming language	C (C99 standard)
GUI framework	GTK-3
Build system	GNU Make (Makefile)
Data storage	Plain-text file (data.txt)
Compiler	GCC with -I headers flag
Version control	Git
Platform	Linux (Ubuntu / Debian)
Module	Algorithms and Data Structures — ALSDD
Academic year	2025 / 2026

2. Project Structure

The project follows a modular directory layout that separates header declarations, source implementations, data, and interface code into dedicated locations.

```
ALSDD_project/
├── main.c           - Entry point and main menu loop
├── gui.c            - GTK-3 graphical user interface
├── Makefile         - Build rules for terminal and GUI targets
├── data.txt         - Dataset of Algerian personalities and events
├── headers/
│   ├── types.h      - Shared struct definitions (TList, TStack, TTree, TQueue)
│   ├── LL_and_queues.h - Linked list and queue function declarations
│   ├── stacks.h     - Stack function declarations
│   ├── BST.h        - Binary search tree function declarations
│   └── recursion.h  - Recursive algorithm declarations
└── sources/
    ├── LL_and_queues.c - Linked list and queue implementations
    ├── stacks.c        - Stack implementations
    ├── BST.c           - BST implementations
    └── recursion.c     - Recursive algorithm implementations
```

2.1 Build System

The Makefile provides four targets:

- `make all` — compiles both the terminal binary (`alsdd`) and the GUI binary (`gui`).
- `make run` — builds everything then immediately launches the GUI.
- `make terminal` — builds and runs the console interface directly.
- `make clean` — removes compiled binaries.

```
gcc main.c sources/*.c -I headers -o alsdd
gcc gui.c sources/*.c -I headers $(shell pkg-config --cflags --libs gtk+-3.0) -o gui
```

3. Data Model

3.1 Shared Struct Definitions (types.h)

All four modules share the same four struct types defined in `headers/types.h`. Each node holds the same four data fields — name, date of birth, date of death, and a definition — but the pointer structure differs per data structure.

```
typedef struct TList {
    /* doubly-linked list node */
    char name[150]; char DoB[30]; char DoD[30]; char definition[2048];
    struct TList *next; struct TList *prev;
} TList;

typedef struct TQueue { ... struct TQueue *next; } TQueue; /* singly-linked, FIFO */
typedef struct TStack { ... struct TStack *next; } TStack; /* singly-linked, LIFO */
typedef struct TTree { ... struct TTree *left; struct TTree *right; } TTree; /* BST */
```

3.2 Data File Format

Each line in `data.txt` encodes one personality or event. Personalities include a date range in curly braces followed by `'='`; events use `':'` instead:

```
Krim Belkacem {14/12/1922-18/10/1970}= Military and political leader of the FLN ...
Battle of Algiers begins {07/01/1957}: FLN launches urban guerrilla campaign ...
```

The parser in `getPersonality()` extracts the name (before `'{'`), date range (inside `'{...}'`), and definition (after `'='`) and builds the doubly-linked list used by all modules.

Field	Location in line	Max size
name	Characters before <code>'{'</code>	150 chars
DoB	Inside <code>'{...}'</code> , before <code>'-'</code>	30 chars
DoD	Inside <code>'{...}'</code> , after <code>'-'</code>	30 chars
definition	Everything after <code>'='</code>	2048 chars

4. Module Descriptions

4.1 Module 1 — Linked Lists & Queues

Operates on the doubly-linked `TList` built from `data.txt` at startup. Provides display, sorting, searching, CRUD, queue conversion, and bidirectional merging.

#	Operation	Description
1	Display all	Print every personality in list order
2	Sort by name	Alphabetical bubble sort (swaps data, not pointers)
3	Sort by name length	Ascending sort on strlen(name)
4	Sort by birth year	Ascending sort on extracted year from DoB
5	Search by DoB	Return all nodes matching a given date of birth
6	Search by DoD	Return all nodes matching a given date of death
7	Palindrome names	Filter nodes whose name reads the same forwards and backwards
8	Similar by date string	Find personalities with a date string matching a query
9	Find by exact date	Search by day, month, and year separately
10	Add personality	Insert new node and persist to file
11	Add event	Append a new event entry
12	Update personality	Modify definition, DoB, DoD for a named node
13	Delete personality	Remove named node and update the file
14	Merge first nodes	Merge head nodes of two lists into a bidirectional structure
15	Name queue (sName)	Convert name fields into a TQueue
16	Age queue (ageP)	Convert DoB fields into a TQueue ordered by age
17	Full queue (toQueue)	Convert entire merged list into a TQueue

4.2 Module 2 — Stacks

Converts the merged TList into a TStack (LIFO) at entry. Implements all standard stack operations plus several dataset-specific analytical functions.

#	Operation	Description
1	Display stack	Print all stack entries from top to bottom
2	Search by name	Linear search for a personality by name
3	Sort alphabetically	Alphabetical sort of the stack by name
4	Sort by definition length	Sort entries by word count of their definition
5	Split short/long events	Partition into two stacks by definition length
6	Add personality	Push a new personality onto the stack
7	Delete personality	Remove a named entry from the stack
8	Update personality	Update definition/dates for a named entry
9	Shortest definition	Return the personality with the fewest definition words
10	Continuous events	Find entries with overlapping date ranges
11	Check if killed	Test if a personality's definition contains death-related keywords
12	Reverse stack (recursive)	Reverse the stack using only recursive calls
13	Stack to sorted queue	Convert and return entries as a sorted TQueue

#	Operation	Description
14	Stack to bidirectional list	Convert and return entries as a doubly-linked TList

4.3 Module 3 — Binary Search Tree (BST)

Reads data.txt at runtime and inserts each personality alphabetically by name into a TTree. BST ordering enables $O(\log n)$ lookup on a balanced tree. A balance-and-rebuild routine (collectInOrder + buildBalanced) is also provided.

#	Operation	Description
1	In-order traversal	Alphabetically sorted output (left → root → right)
2	Pre-order traversal	Root → left → right traversal
3	Post-order traversal	Left → right → root traversal
4	Search by name	BST search — $O(\log n)$ on a balanced tree
5	Add name	Insert a new personality into the correct BST position
6	Delete name	Remove a node (handles 0, 1, and 2 children)
7	Update name	Modify definition/dates for a node in the tree
8	Height and size	Compute and display tree height and total node count
9	Lowest Common Ancestor	Find the LCA of two named nodes
10	Count in birth-year range	Count nodes whose birth year falls in [low, high]
11	In-order successor	Find the in-order successor of a named node
12	Mirror tree	Swap left and right children recursively throughout the tree
13	Check balance	Verify the AVL balance condition on the entire tree
14	Merge trees	Merge the current tree with a fresh copy built from the file

4.4 Module 4 — Recursion

Demonstrates pure recursive algorithms applied to data.txt and to user-supplied strings. All functions avoid loops and rely exclusively on recursive calls.

#	Operation	Description
1	Count occurrences	Recursively count lines in data.txt containing a given name
2	Remove occurrences	Recursively remove all matching lines, writing result to temp.txt
3	Replace occurrences	Recursively update date fields for all matching lines
4	Name permutations	Print all permutations of a name string (recursive backtracking)
5	All subsequences	Generate and print all subsequences of a word recursively
6	Overlapping date range	Find the longest overlapping period between two date strings
7	Distinct subsequences	Count distinct subsequences of a string recursively
8	Palindrome check	Verify whether a string is a palindrome using recursion

5. Graphical User Interface (gui.c)

In addition to the terminal interface, the project ships a complete GTK-3 graphical frontend in `gui.c`. The GUI wraps the same `alsdd` binary: it collects user inputs through dialog boxes, feeds them to the backend via a redirected stdin (temporary file on Linux), and displays the captured output in a scrollable text view.

5.1 Window Layout

- Header bar — displays the application title.
- Left sidebar (240 px) — shows the four module buttons on the main menu, or the scrollable action list for the selected module.
- Right output pane — a read-only `GtkTextView` with monospace font displaying program output after each run.
- Status bar — one-line message (Ready / Running... / Done.).

5.2 Interaction Flow

- User clicks a module button on the main menu.
- The sidebar updates to show the numbered action list for that module, with a Back button.
- If the action requires parameters, a modal dialog appears with labeled entry fields.
- On confirmation, the GUI assembles the stdin string (module number, action number, field values, then '0 0' to exit), writes it to a temp file, and runs `./alsdd < tmpfile 2>&1`.
- The captured output is displayed in the output pane and the status bar shows Done.

5.3 Working Directory Fix

`fopen("data.txt", "r+")` in `main.c` uses a relative path. If `gui` is launched from a directory other than the project folder, `data.txt` is not found. The fix in `gui.c` resolves the real executable path at startup via `/proc/self/exe` and calls `chdir()` to that directory before every spawn:

```
ssize_t len = readlink("/proc/self/exe", resolved, sizeof(resolved)-1);
char *dir = dirname(resolved);
strncpy(g_exe_dir, dir, sizeof(g_exe_dir)-1);

// Before every spawn:
if (g_exe_dir[0]) chdir(g_exe_dir);
```

6. Key Implementation Notes

6.1 File Parsing

The data file is opened once in `main()` with `fopen("data.txt", "r+")`. Both `getPersonality()` and `getDatePersonality()` call `rewind(f)` before scanning, so they always start from the beginning. The two resulting lists are then zipped into a single merged doubly-linked list passed to the Stacks module.

6.2 Memory Management

Every node is allocated with `malloc()` and zero-initialised with `memset()`. At program exit, `main()` walks all three top-level lists (`s`, `a`, `merged`) and `free()`s each node. BST and Stack modules allocate their own nodes internally; OS cleanup handles them on exit in the current version.

6.3 Sorting Strategy

All sorting in the Linked List and Stack modules uses in-place bubble sort that swaps data fields (via `swapData`) rather than relinking pointers. This simplifies pointer logic at the cost of copying up to 2048-byte definition strings per swap. The BST maintains ordering through standard BST insertion.

6.4 Recursion Module Limitations

The file-manipulation recursive functions (`removeOccurence`, `replaceOccurence`) write to a hardcoded `temp.txt` file. Each recursive call opens and closes this file independently, which can cause incomplete writes on large datasets. In a production version, the temp file path should use `mkstemp()` and a final `rename()` should atomically replace `data.txt`.

6.5 BST Balance

The BST is built by inserting nodes in file order. If the file is already sorted, the tree degenerates to a linked list ($O(n)$ search). The `buildBalanced()` function — using `collectInOrder()` then recursive midpoint insertion — can rebuild a height-balanced tree with $O(\log n)$ lookup.

7. How to Build and Run

7.1 Prerequisites

- GCC (C99 support)
- GTK-3 development libraries: `sudo apt install libgtk-3-dev`
- GNU Make

7.2 Build Steps

```
cd ALSDD_project/

make run          # execute in gui

make terminal     # execute in terminal
```

8. Team Members and Conclusion

8.1 Project Team

#	Full Name	Group
1	FOUNAS Ouail	A4
2	KEBBICHE Anes	B1

8.2 Conclusion

This project provided practical experience in implementing the four core data structures studied in the ALSDD module — doubly-linked lists, stacks, binary search trees, and recursive algorithms — within a coherent, real-world application.

By applying each structure to the same Algerian historical dataset, we could directly compare their strengths: the linked list offers flexible sequential access and easy modification; the stack demonstrates LIFO behaviour and recursive reversal; the BST enables fast ordered lookup; and the recursion module highlights the elegance and trade-offs of pure recursive approaches on file data.

The GTK-3 GUI makes the application accessible to non-technical users and shows how a C backend can be cleanly wrapped in a graphical front-end without modifying the underlying logic.

Overall, this project deepened our understanding of data structure trade-offs, C memory management, file I/O, and GUI programming — skills directly applicable to systems and security engineering at NHSCS.

The Project Team

NHSCS — Algiers, Academic Year 2025/2026